Git（ギット）

ファイルのバージョン管理をするための仕組み（ツール）

Linux を作ったリーナスが 2005 年に開発した分散型バージョン管理システム

・変更履歴を記録していつでも過去に戻せる

・誰が「いつ」「どこを」「どう変えたか」を管理できる

基本的にはコマンドで操作（例）
git init →　リポジトリ作成（監視すべき対象フォルダの設定）
git add ファイル名　→　対象ファイルをステージング
git commit -m “メッセージ”　→　ステージングしたファイルをコミット

git status　→　現在のリポジトリの状態確認
git log --oneline　→　一行でコミット履歴を確認
git reset --soft(--hard) コミット番号　→　指定したコミット番号のバージョンに戻す
rm -fr .git　→　リポジトリの削除　（対象フォルダの監視をやめる：管理データは消滅）

GitHub（ギットハブ）

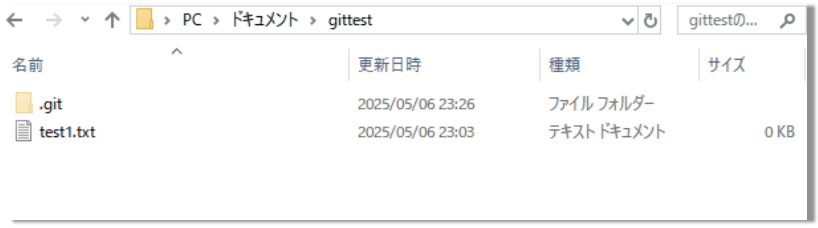
類似サービス：
GitLab、BitBucket、Assembla、Phabricator

Git で管理しているファイルを、インターネット上に保存・共有できるサービス

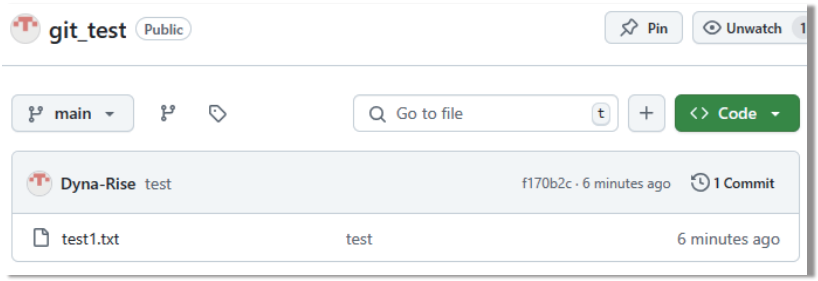
クリス・ワンストラス（Chris Wanstrath）らが設立した GitHub 社（現：Microsoft 傘下）が提供

・作ったプログラムをチームで共有したり、バックアップに使ったりできる

コマンドで git と連携することが可能
git remote add origin “リポジトリの URL”　→GitHub のリポジトリ (リモート）の登録
git push origin main　→　リモート (origin) へローカル (main) の情報をプッシュ



ローカルリポジトリ
パソコンに保存してあるフォルダ



リモートリポジトリ
クラウド上 (GitHub サービス上)
に用意した領域（リポジトリ）

SourceTree（ソースツリー）

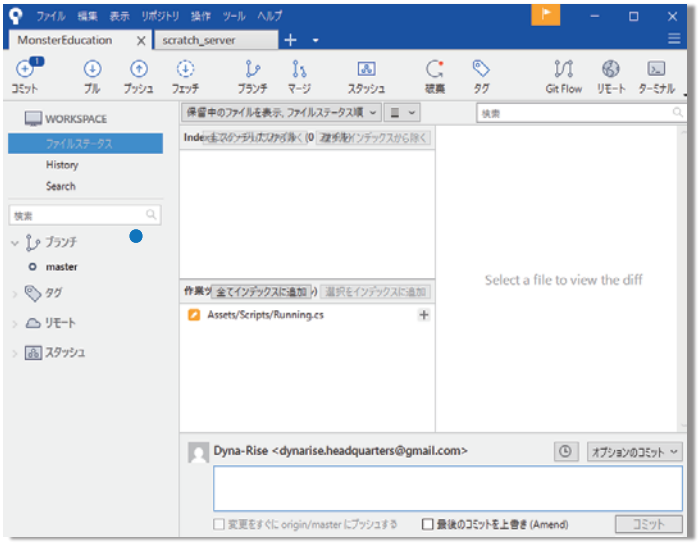
類似サービス：
GitKraken、GitHub Desktop、Fork、TortoiseGit

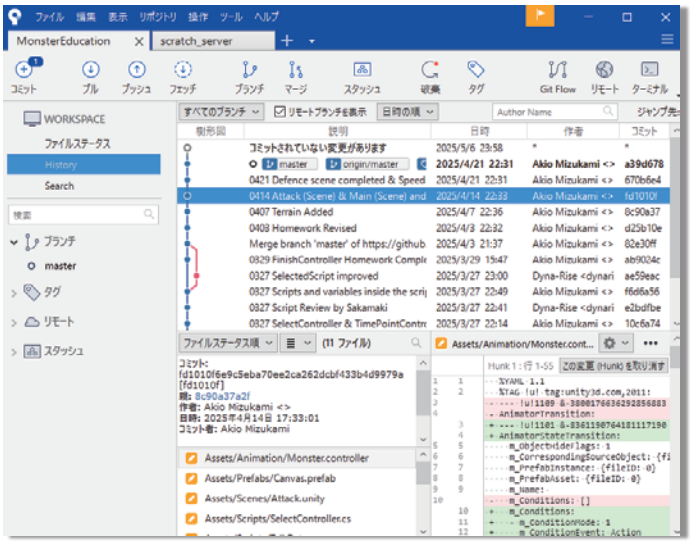
Git を操作しやすくするサポートアプリ

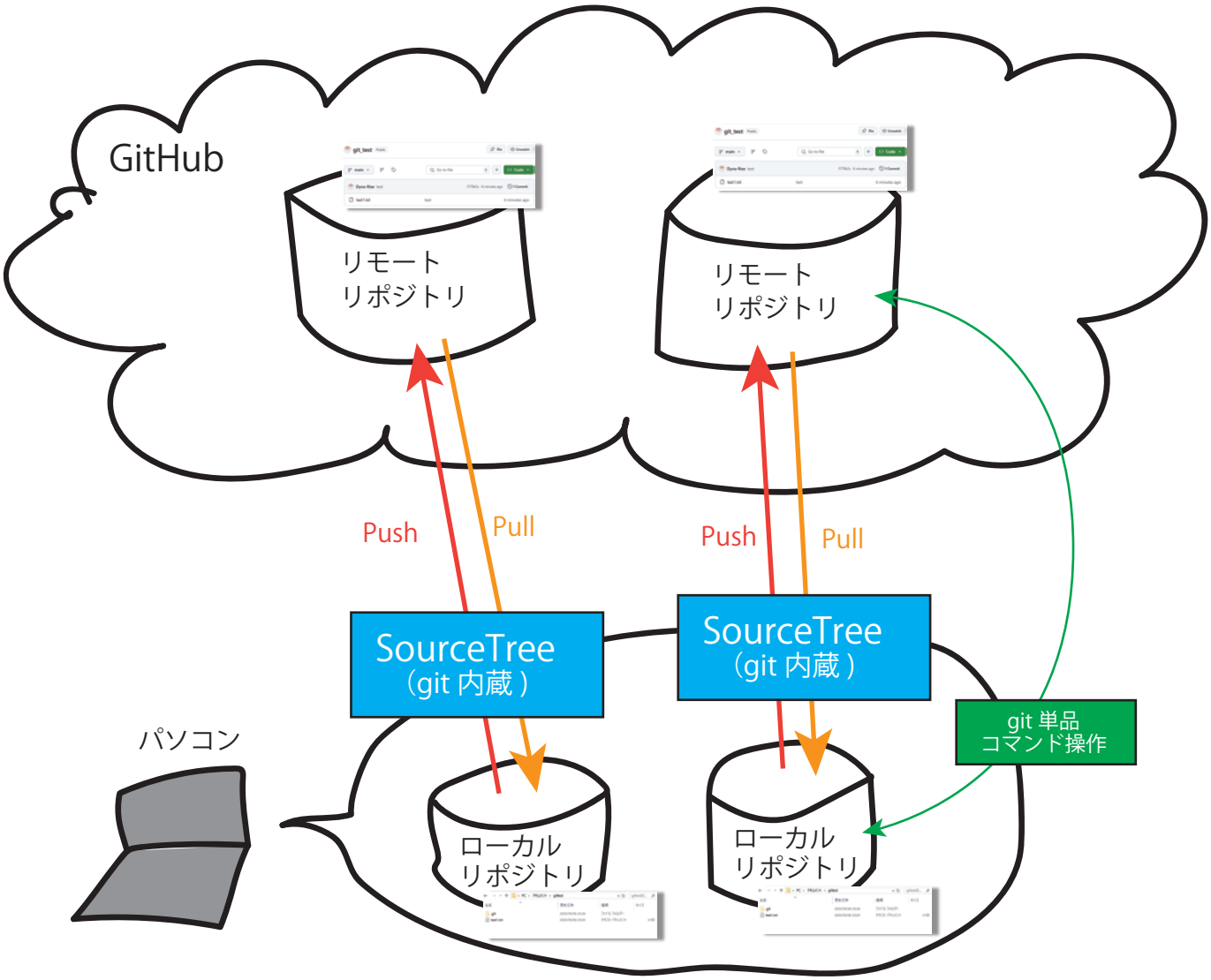
Steve Streeting 氏の個人開発が、2011 年に Atlassian が買収

・本来コマンド操作による git とクラウドとの連携を GUI で成立させるためのアプリ

・ステージング段階で複数のファイル選定が用意







◆ リポジトリ（Repository）
「管理対象のデータのかたまり / または管理対象のデータを保存する領域」
・リポジトリの中でファイルの履歴がすべて追えるようになる
・クラウドのリポジトリをリモート、パソコンのリポジトリをローカルとして連携

◆ コミット（Commit）
「変更をひとまとまりで記憶する作業」
・特定の状況の " スナップショット " をとる＝記録に残る
・コミットには必ず「メッセージ」（何を変更したかのメモ）をつける

◆ ステージング（Stageing）
「コミットのためのファイルの選定」
・まずは選定したファイルをステージング→コミットで状況を変更を記録する

◆ プッシュ（Push）
「コミットで記録した変更をリモート（クラウド）に送る」
・プッシュしてはじめて他者に変更状況が共有される

◆ プル（Pull）
「リモート（クラウド）上の変更差分を自分の手元（ローカル）に取り込む」
・他の人の作業を自分のファイルに反映させることができる
・プルしてから続きの編集をしないと、共有相手の作業と差異が生じトラブルになる

◆ フェッチ（Fetch）
「リモート（クラウド）上にどんな変更差分があるか確認する」
・フェッチはあくまでリモートの最新状況の確認のみ
・実際に反映するにはマージやプルが必要

◆ ブランチ（Branch）
「バージョンを枝分かれさせる (本筋から分岐したサブをつくる)」
・本線（main、origin）を崩さずに別の可能性として新しい機能追加や修正を試せる
・作業が完成したら本線（main、origin）にマージ (機能を合流) する
→(例) 新機能「会員登録システム」を連動させたいが完成まで時間がかかる
すぐに main に反映できないのでブランチで枝分かれさせた Ver. で開発を進める
ブランチ ver. で問題なければ main(現状 ver.) にマージして新機能追加を完遂

◆ マージ（Merge）
「枝分かれして作業した別バージョンの差異を本線にまとめる」
・マージするとき内容がぶつかったら
（同じ場所を違う人が書いていたら）「コンフリクト（衝突）」が起きてしまう
・コンフリクトは手動で調整 (どちらが正しいのか選定) する